

System Architecture for Real-Time AI Service Markets

University of Oulu, Faculty of ITEE, CSE Research Unit

Contents

1. Architectural Overview	2
Design Principles	3
2. The Integrator: Dual Role	4
2.1 Encapsulator (Architectural Role, Paper 1)	4
2.2 Operator (Mechanism-Design Role, Paper 2)	4
3. Integrator Internal Architecture	4
3.1 Sub-DAG Manager	4
3.2 Internal Scheduler	5
3.3 Slice Interface	5
3.4 Mechanism Executor	5
3.5 Resource Procurer	5
3.6 Governance Interface	5
4. Inter-Component Communication	6
4.1 Agent-to-Integrator (Level 1)	6
4.2 Integrator-to-Integrator (Cross-Domain)	6
4.3 Integrator-to-Local-Marketplace (Level 2)	6
4.4 Integrator-to-Governance	7
5. Overall System Architecture	7

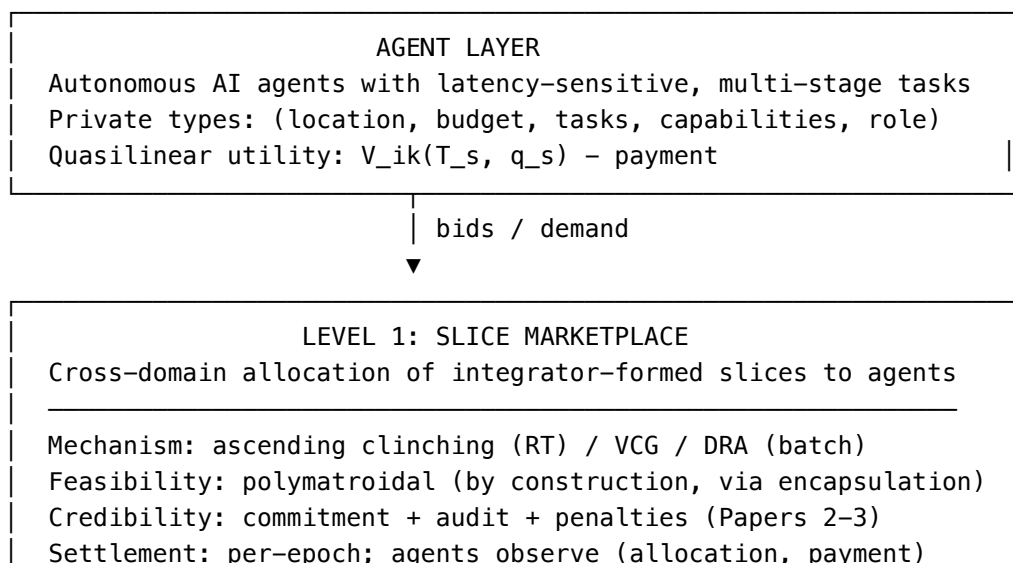
6. Major Implementation Challenges	8
6.1 The Max-Flow Abstraction Is Lossy	8
6.2 GS Valuations Are Fragile	9
6.3 Integrator Credibility (Who Watches the Integrator?)	9
6.4 The Broadcast Channel Assumption	10
6.5 Real-Time Latency Budget	10
6.6 Dynamic Topology	11
6.7 Agent-Learner Feedback Loop	11
6.8 Privacy-Audit-Latency Triangle	12
6.9 Recursive Credibility	12
6.10 Empirical Calibration Gap	12
6.11 Penalty Calibration for Contextual Adversaries	13
7. Summary: Challenge Severity and Mitigation Status	13

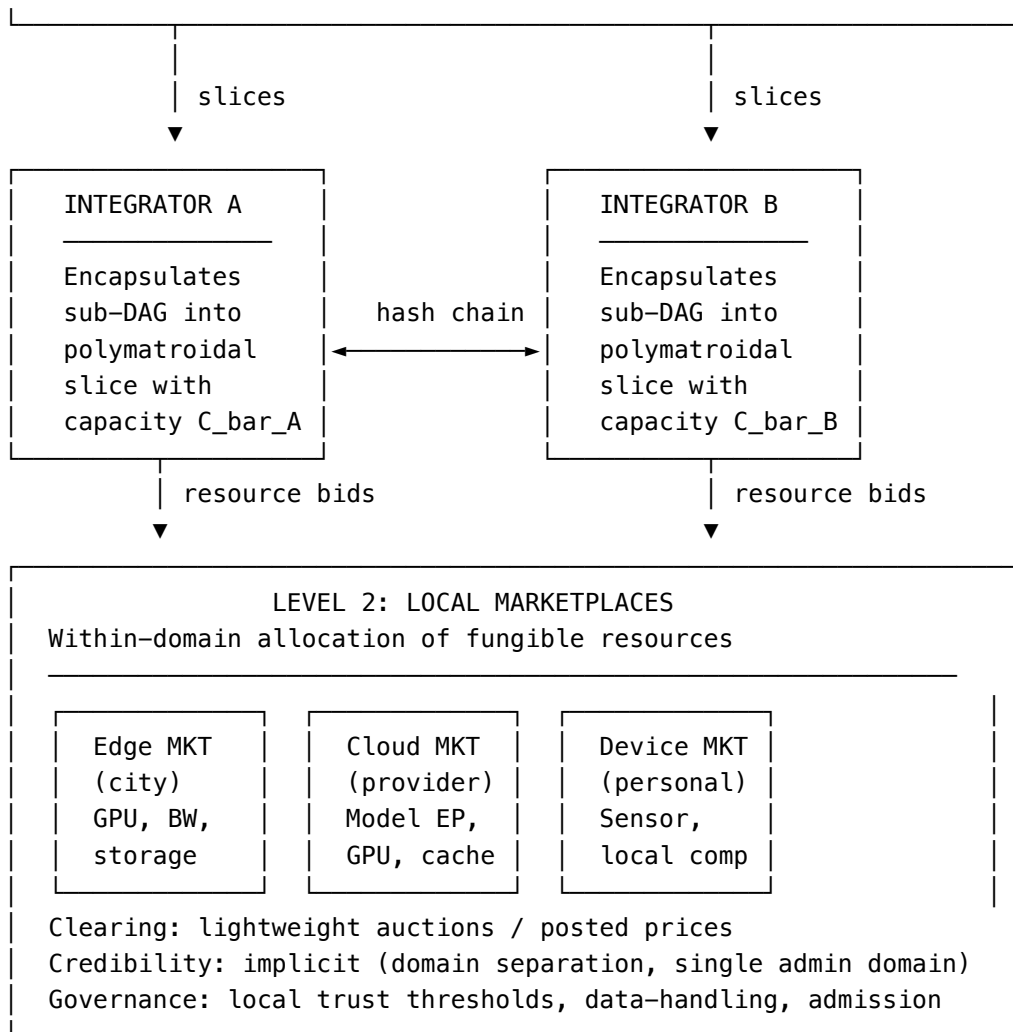
Analysis of the overall system architecture, the integrator component, and major implementation challenges, based on the theory in the three IEEE TSC papers.

Companion document to [governance-architecture.md](#).

1. Architectural Overview

The trilogy defines a three-layer hybrid architecture for managing real-time AI service economies across the device-edge-cloud continuum.





Design Principles

1. **Topology determines tractability** (Paper 1). Tree and series-parallel DAGs yield polymatroidal feasibility regions, enabling efficient allocation and DSIC mechanisms. Arbitrary DAGs break all three guarantees.
2. **Encapsulation restores structure** (Paper 1, Proposition 3). Integrators absorb complementarities inside domain boundaries and expose substitutable slice interfaces. If the quotient graph (after contraction) is tree or SP, the agent-facing region is polymatroidal.
3. **Credibility requires commitment** (Paper 2). No sealed-bid mechanism on a non-modular polymatroid is simultaneously revenue-optimal, DSIC, and credible. Broadcast channels, domain separation, or blockchain deposits resolve this.
4. **Adversarial resilience requires layered governance** (Paper 3). Coalition audit, adaptive penalties, dynamic certification, and market discipline are complementary layers; removing any one creates an exploitable gap.

2. The Integrator: Dual Role

The integrator is the central architectural component. Papers 1 and 2 reveal that it simultaneously fills two roles that create a fundamental tension.

2.1 Encapsulator (Architectural Role, Paper 1)

The integrator manages a connected sub-DAG of the service-dependency graph:

Inputs: - Internal sub-DAG G_j with node capacities C_v - Raw resources procured from Level 2 local marketplaces - Governance constraints (data-locality, trust thresholds, jurisdictional tags)

Internal processing: - Max-flow computation: $C_{\bar{j}} = \text{maxflow}(G_j)$ using Ford-Fulkerson or push-relabel - Node-splitting for node capacities: $v \rightarrow (v_{\text{in}}, v_{\text{out}})$ with edge capacity C_v - Super-source/super-sink construction for multi-entry/exit sub-DAGs - Internal scheduling optimisation (efficiency factor: 0.75 for SP, 0.85 for entangled) - Governance constraint enforcement (capacity partitioning, trust-gated access)

Output: - A single composite slice with: - Scalar capacity $C_{\bar{j}}$ (max-flow of internal sub-DAG) - Deterministic latency T_s - Quality q_s - Fixed attributes within each mechanism epoch

2.2 Operator (Mechanism-Design Role, Paper 2)

The integrator also executes the allocation mechanism within its domain:

Functions: - Receives agent bids for slices - Computes allocation and payments (VCG, ascending clinching, DRA) - Observes all bids (information advantage over agents) - Could deviate: capacity misreporting, price manipulation, discriminatory allocation

The tension: The same entity that (a) knows its true internal capacity and (b) sets the mechanism rules also (c) has a revenue incentive to deviate from faithful execution. This is the credibility gap that Paper 2 formalises (Theorem 1: credibility trilemma) and Papers 2-3 resolve via commitment, audit, and penalties.

3. Integrator Internal Architecture

Based on the theory across all three papers, the integrator requires these internal subsystems:

3.1 Sub-DAG Manager

Function: Maintains the internal service-dependency graph, computes max-flow, handles topology changes.

- **Max-flow engine:** Polynomial-time computation (Ford-Fulkerson / push-relabel), recomputed periodically as internal conditions change
- **Node-splitting:** Converts node-capacitated DAGs to edge-capacitated networks
- **Super-source/sink:** Handles multi-entry/exit sub-DAGs
- **Topology monitor:** Detects when internal services join, leave, or change capacity; triggers max-flow recomputation

Recomputation frequency: At minimum once per mechanism epoch. More frequent recomputation enables tighter capacity tracking at the cost of compute overhead.

3.2 Internal Scheduler

Function: Allocates internal resources to fulfill slice commitments. This is where encapsulation creates value: centralised scheduling absorbs complementarities that would destabilise market-based allocation.

- Routes tasks through the internal sub-DAG
- Manages latency-throughput trade-offs (the Pareto frontier that the scalar max-flow abstraction simplifies)
- Enforces internal governance constraints
- Produces the efficiency factor (0.75-0.85 in simulation)

3.3 Slice Interface

Function: Exposes the simplified capacity abstraction to the Level 1 marketplace.

- Publishes (C_{bar_j} , T_s , q_s) per epoch
- Attributes are fixed within each epoch (required for GS property; see Lemma 1 condition (c))
- Must faithfully summarise internal capacity (assumption E2 in Paper 2; violation is a deviation)

3.4 Mechanism Executor

Function: Runs the allocation mechanism for its domain at Level 1.

- Ascending clinching auction (real-time, with broadcast)
- VCG (sealed-bid, requires commitment for credibility)
- DRA (batch, with blockchain deposits)
- Commitment interface: publishes hash commitments, participates in cross-hashing with adjacent integrators

3.5 Resource Procurer

Function: Bids at Level 2 local marketplaces for raw resources to fulfil slice commitments.

- Translates slice demand into resource-level bids (compute, bandwidth, storage, inference endpoints)
- May bid at multiple local marketplaces across domains
- Manages cross-domain resource assembly

3.6 Governance Interface

Function: Interfaces with the governance stack (see governance-architecture.md).

- Reports bid transcripts to FCA (encrypted, per-epoch)

- Publishes commitment hashes and receives adjacent integrators' hashes
- Maintains slashing deposit with Penalty Engine
- Responds to certification signals (warning, decertification, recertification)
- Enforces governance capacity constraints (X_{gov}) within its domain

4. Inter-Component Communication

4.1 Agent-to-Integrator (Level 1)

Signal	Direction	Content	Frequency
Slice advertisement	Integrator -> Agent	$(C_{bar_j}, T_s, q_s, price)$	Per epoch
Bid	Agent -> Marketplace	$b_i = V_{ik}(T_s, q_s)$	Per epoch
Allocation + payment	Marketplace -> Agent	(x_i, p_i)	Per epoch
Demand/exit broadcast	Agent -> All (broadcast)	Demand at current price	Per price step (ascending)

4.2 Integrator-to-Integrator (Cross-Domain)

Signal	Direction	Content	Frequency
Hash commitment	Bidirectional (adjacent)	$H_j = Hash(bids, alloc, payments, \{H_k\})$	Per epoch
Coarse coordination	Bidirectional	Congestion indicator, residual capacity	Periodic
Coalition detection	Via FCA	Discrepancy flags	Per epoch

4.3 Integrator-to-Local-Marketplace (Level 2)

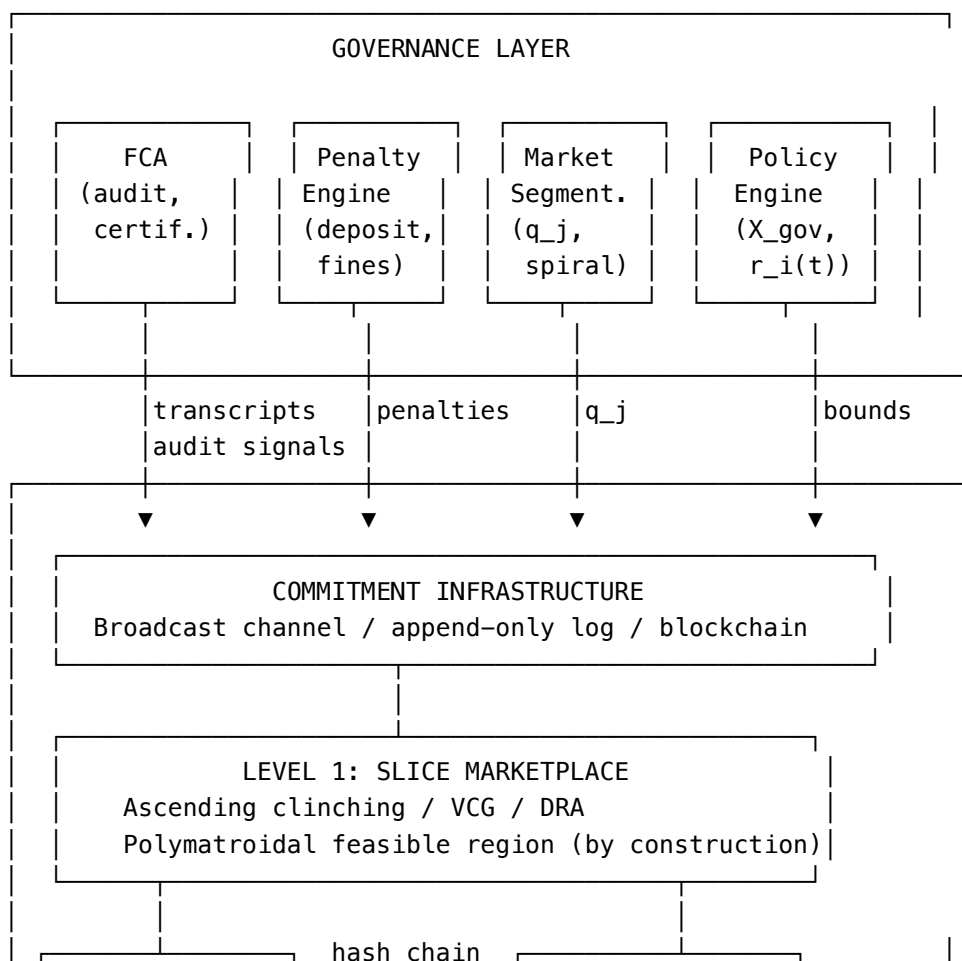
Signal	Direction	Content	Frequency
Resource bid	Integrator -> L2 Market	Demand for compute/BW/storage	Per epoch
Resource allocation	L2 Market -> Integrator	Allocated resources + price	Per epoch
Congestion signal	L2 Market -> Integrator	Queue depth, utilisation ratio	Periodic
Trust/policy tags	Integrator -> L2 Market	Cross-domain governance metadata	On change

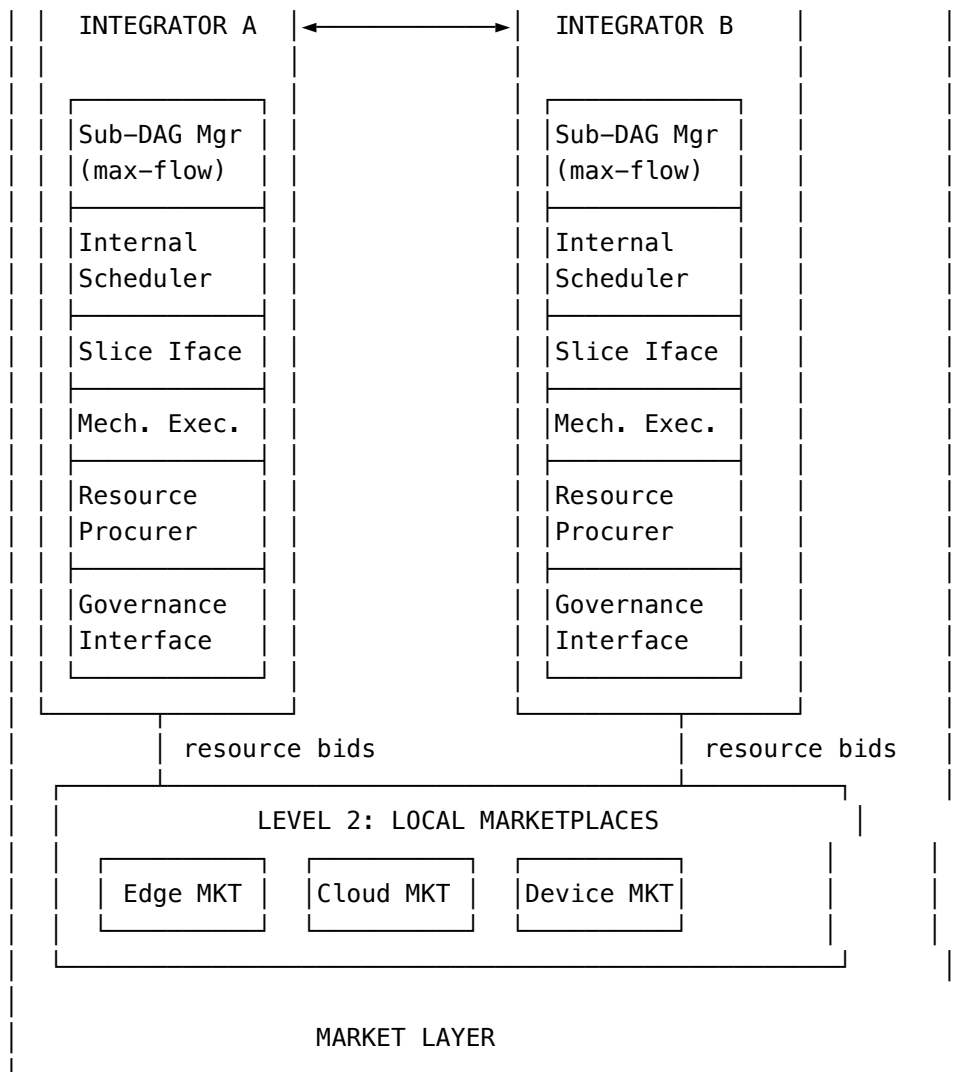
4.4 Integrator-to-Governance

Signal	Direction	Content	Frequency
Bid transcript	Integrator -> FCA	Encrypted (bids, alloc, payments)	Per epoch
Audit signal	FCA -> Integrator	z_t in {compliant, anomalous}	Per epoch
Certification state	FCA -> Integrator	{certified, warning, decertified}	On change
Deposit status	Penalty Engine <-> Integrator	Bond balance, slashing events	On change
Credibility quality	FCA -> Market Seg. Controller	q_j (posterior-derived)	Per epoch

5. Overall System Architecture

Combining the market structure, integrator internals, and governance stack from all three papers:





6. Major Implementation Challenges

The challenges below are ordered roughly from most fundamental (structural) to most practical (engineering).

6.1 The Max-Flow Abstraction Is Lossy

Source: Paper 1, Proposition 3 scope limitations.

The integrator summarises its internal sub-DAG as a scalar capacity $C_{\text{bar}_j} = \text{maxflow}(G_j)$. This abstraction is valid when the integrator exports a single, homogeneous slice type. It breaks in three identified ways:

(a) Multi-output integrators. When an integrator exports multiple service types, C_{bar_j} generalises to a vector-valued capacity, requiring each integrator's external feasibility set to itself be polymatroidal. This is a strictly stronger structural condition.

(b) Latency-throughput Pareto frontiers. Internal scheduling involves non-trivial trade-offs (e.g., routing through a fast-but-narrow path vs a slow-but-wide one). The scalar max-flow

abstraction loses this decision-relevant information. Agents observe only T_s (a single latency point), not the frontier.

(c) Shared leaf nodes. When multiple integrators share underlying infrastructure (e.g., a common cloud GPU pool), hidden feasibility constraints couple their capacities externally, violating the independence assumption of Proposition 3.

Implication: Real integrators must either (i) genuinely export a single slice type (limiting architectural flexibility), (ii) implement internal polymatroidal feasibility (nested complexity), or (iii) accept that the theoretical guarantees degrade gracefully but not perfectly.

6.2 GS Valuations Are Fragile

Source: Paper 1, Lemma 1 conditions.

The gross-substitutes property that enables Walrasian equilibrium, efficient computation, and DSIC holds only if:

- Each agent requires at most one slice per task (unit demand)
- Tasks are additively separable (no cross-task complementarity)
- Slice attributes (T_s , q_s) are fixed within each epoch

Any of the following breaks GS: shared agent-side budgets across tasks, cross-task latency coupling (e.g., total pipeline latency matters, not per-stage), or endogenous quality changes within an epoch.

Implication: Realistic AI agent workloads often exhibit cross-task coupling (a multi-stage inference pipeline where total end-to-end latency determines value, not individual stage latencies). Encapsulation helps (an integrator can bundle the entire pipeline into one slice), but this pushes complexity inward and may require very coarse-grained slicing.

6.3 Integrator Credibility (Who Watches the Integrator?)

Source: Paper 2, assumption E2; Paper 3, recursive credibility.

The integrator has four deviation channels:

1. **Capacity misreporting:** understating C_{bar_j} to create artificial scarcity
2. **Price manipulation:** computing payments differing from the prescribed mechanism
3. **Selective information revelation:** sharing bid information with affiliated agents
4. **Discriminatory allocation:** favouring certain agents off-equilibrium

Paper 2 addresses channels 2-4 via commitment (broadcast ascending auction makes deviations detectable). Paper 3 addresses multi-integrator coalitions via cross-market audit.

But channel 1 remains open. The integrator privately knows its true max-flow. No external party can verify C_{bar_j} without full visibility into the internal sub-DAG. Paper 2 lists “integrator credibility theory” (faithful max-flow reporting) as an explicit open problem (assumption E2). Competition among integrators (Proposition 2 of Paper 2) provides indirect discipline, but only for pricing, not capacity reporting.

Implication: A practical system needs either (a) independent capacity verification (possibly via redundant monitoring or benchmark probing), (b) incentive-compatible capacity reporting mechanisms (designing a mechanism where truthful capacity reporting is a dominant strategy), or (c) competitive pressure from alternative integrators offering substitutable slices.

6.4 The Broadcast Channel Assumption

Source: Paper 2, Theorem 2 assumptions B1-B2.

The ascending clinching auction's credibility depends on two assumptions:

- **(B1)** Agents broadcast their own demand/exit messages *directly* to all participants (not relayed by the operator)
- **(B2)** The rank function f (polymatroid structure) is publicly known

B1 requires independent communication infrastructure. In practice, this means a peer-to-peer broadcast overlay, an independent message bus, or a blockchain. All options add latency, which conflicts with the 100-200ms budget for real-time AI services.

B2 requires topology transparency. The polymatroid rank function encodes the capacity constraints of the service-dependency DAG. Publishing it reveals infrastructure topology and capacity, which may be proprietary. ZKP-based alternatives exist in principle but add computational overhead.

Implication: The most theoretically appealing credibility mechanism (ascending clinching with broadcast) has the hardest deployment requirements. Domain-separated neutral exchanges avoid B1-B2 but require institutional trust. The system architect must choose based on the deployment context.

6.5 Real-Time Latency Budget

Source: All three papers; Paper 2 deployment hierarchy.

The mechanism must complete within the epoch's latency budget. Competing demands:

Operation	Typical latency	Budget pressure
Max-flow recomputation	$O(V * E)$ per integrator	Per-epoch
Ascending auction (price steps)	Proportional to $V_{\max} * K$	Per-epoch
Broadcast round-trip	Network-dependent	Per price step
FCA cross-market audit	Proportional to m integrators	Post-epoch
Hash chain computation	$O(m)$ hashes	Post-epoch
Bayesian posterior update	$O(1)$ per integrator	Post-epoch

For real-time AI services with 100-200ms interactive deadlines, the ascending auction with broadcast may consume most of the budget. The FCA audit and hash chain can run post-epoch (non-blocking), but their results inform the *next* epoch's governance state (one-epoch detection delay).

Implication: The system may need to separate the *allocation path* (latency-critical: ascending auction or posted prices) from the *audit path* (latency-tolerant: FCA verification, hash

chains, penalty enforcement). Allocation runs at real-time speed; governance catches up asynchronously.

6.6 Dynamic Topology

Source: Paper 1, open challenge; Paper 3, non-stationarity.

The theory assumes the service-dependency DAG G_{res} is fixed within each epoch. In practice, AI service pipelines recompose at runtime: new services deploy, existing ones fail, agents switch between pipeline configurations.

Three sub-problems:

(a) Intra-epoch changes. If a service fails mid-epoch, the integrator's capacity changes, but the mechanism has already committed to allocations. Requires either epoch-aligned failure handling or a rollback/reallocation mechanism.

(b) Cross-epoch structural changes. If the quotient graph changes topology (e.g., from tree to entangled due to a new cross-domain dependency), the polymatroidal guarantee may break between epochs. The system must detect this and either (i) refuse the topology change, (ii) deploy additional integrators to restore structure, or (iii) degrade gracefully.

(c) Integrator lifecycle. Integrators may join or leave the system, changing the competitive landscape and the quotient graph structure. The governance layer (certification, deposits) must handle integrator churn.

Implication: The architecture needs a topology monitor that validates the polymatroidal property of the quotient graph before each epoch and triggers restructuring (re-partitioning, new integrator deployment) when it's violated.

6.7 Agent-Learner Feedback Loop

Source: Paper 3, Discussion limitations.

The theory treats two phenomena independently:

- **Theorem 2** (learning deterrence) assumes i.i.d. agent bids across rounds
- **Proposition 2** (strategic agents) models agents exiting in response to credibility failures

In reality, these interact: operator deviations trigger agent exit, which changes the bid distribution, which alters the operator's learning signal. This feedback loop creates non-stationarity that neither result captures alone.

Concrete risk: A learning operator explores deviations, triggering a wave of agent departures. The depleted agent pool changes the optimal deviation strategy. The operator's learner may converge to a sub-optimal policy calibrated to the depleted market, or oscillate between exploitation and recovery phases.

Implication: The monitoring layer (Proposition 1's Bayesian type tracking) must be robust to the non-stationarity created by this feedback. The simulation (Exp 11) tests this interaction empirically, but finite-time bounds on welfare loss remain open.

6.8 Privacy-Audit-Latency Triangle

Source: Paper 3, Discussion.

The system faces a three-way tension:

- **Audit requires bid transcripts.** The FCA needs (bids, alloc, payments) from all domains to detect coalition deviations.
- **Privacy requires bid confidentiality.** Agents may not want bids disclosed; integrators resist revealing proprietary pricing.
- **Latency requires speed.** ZKP/MPC for privacy-preserving audit adds computational overhead that may be incompatible with real-time epochs.

State of the art: ZKP-based auction verification exists in the blockchain literature but typically operates on timescales of seconds to minutes, not the 100-200ms required for real-time service markets.

Implication: The system may need to accept a trade-off: full audit (with privacy costs) for high-value cross-domain allocations, and lighter-touch statistical monitoring for latency-sensitive flows. This creates a gap that sophisticated coalitions could exploit by restricting deviations to the lightly-audited fast path.

6.9 Recursive Credibility

Source: Paper 3, Discussion.

The governance stack requires penalty enforcement, but the entity enforcing penalties may itself have incentives to deviate.

Chain of trust: 1. Agents trust the mechanism (if DSIC) 2. DSIC requires credible execution (commitment) 3. Credible execution requires audit (FCA) 4. Audit requires penalty enforcement (Penalty Engine) 5. **Who enforces the Penalty Engine?**

Three resolution paths: - **Smart contracts:** Automatic slashing, but requires blockchain infrastructure (latency cost) - **Regulatory authority:** External legal enforcement, but slow and jurisdiction-dependent - **Multi-party computation:** Distributed governance with no single point of trust, but computationally expensive

Implication: Every practical deployment must ground the trust chain somewhere. The choice of grounding mechanism (blockchain, regulation, consortium agreement) is a deployment-context decision that the theory does not resolve.

6.10 Empirical Calibration Gap

Source: Paper 3, Discussion.

The theory uses stylised parameters:

Parameter	Theory assumes	Reality
Non-modularity gap γ_{ij}	Fixed, known	Unknown; varies by topology and load

Parameter	Theory assumes	Reality
Deviation gain Δ_{bar}	Fixed or state-dependent	Must be estimated from bid distributions
Forgetting factor ρ	Configurable	Requires calibration from transition data
Agent outside option u_0	Homogeneous	Heterogeneous, possibly unknown
Signal accuracy $1-\epsilon$	Fixed	Depends on audit mechanism quality

Implication: Before deployment, each parameter requires either (a) empirical measurement from real edge-cloud service traces (proprietary data), (b) conservative worst-case bounds (which may make the system uneconomical), or (c) adaptive estimation procedures running alongside the mechanism (adding complexity and convergence time).

6.11 Penalty Calibration for Contextual Adversaries

Source: Paper 3, Theorem 2(ii).

State-independent penalties (fixed slashing deposits) suffice for epsilon-greedy and no-regret learners but fail against contextual-bandit adversaries. These adversaries learn to deviate only in states where $\Delta_{\text{bar}}(s) > p_F$, requiring state-dependent penalty schedules.

Implementation requires: 1. An estimator for $\Delta_{\text{bar}}(s)$ conditioned on each observable state (needs historical bid data, may be noisy) 2. A commitment device that credibly promises state-dependent penalties (either smart contract with oracle access or FCA-managed fine schedule) 3. Safety margin $\alpha > 1$ to account for estimation error

Implication: Adaptive penalty estimation is itself a learning problem operating concurrently with the adversary's learning. If the penalty estimator lags behind the adversary's state exploitation, a window of vulnerability opens.

7. Summary: Challenge Severity and Mitigation Status

#	Challenge	Severity	Mitigation in trilogy	Open?
6.1	Lossy max-flow abstraction	High	Single-slice assumption	Yes (multi-output)
6.2	Fragile GS valuations	High	Encapsulation + conditions	Partially
6.3	Integrator capacity credibility	High	Competition (indirect)	Yes (E2 open)
6.4	Broadcast channel deployment	Medium	Three-tier hierarchy	Engineering
6.5	Real-time latency budget	Medium	Allocation/audit separation	Engineering

#	Challenge	Severity	Mitigation in trilogy	Open?
6.6	Dynamic topology	Medium	Per-epoch fixity assumption	Yes
6.7	Agent-learner feedback	Medium	Empirical (Exp 11)	Yes (no bounds)
6.8	Privacy-audit-latency	Medium	Acknowledged as open	Yes
6.9	Recursive credibility	High	Three paths proposed	Yes (not resolved)
6.10	Empirical calibration	Medium	Simulation calibration	Yes (real data)
6.11	Contextual penalty calibration	Medium	Adaptive fines (theory)	Partially

The three highest-severity open challenges (6.1, 6.3, 6.9) share a common theme: **the integrator is trusted more than the theory can verify**. It is trusted to report capacity honestly (6.1/6.3), and the governance layer is trusted to enforce penalties honestly (6.9). Resolving this trust bottleneck, whether through verifiable computation, competitive pressure, or institutional design, is the single most important step toward practical deployment.